

MSc Project 1: Time-Series-Enhanced Predictive Process Monitoring Final Report

Michal Durkalec, Konstantinos Sakkas, and Roman Ležovič

Supervised Process Intelligence (SPI) Lab,
RWTH Aachen University, Aachen, Germany
`office@pads.rwth-aachen.de`

Abstract. Predictive process monitoring uses historical event log data to predict outcomes and remaining times of running process instances. Standard approaches derive features from the event log alone, ignoring temporal resource-load variation. This project tests whether aligned time-series features (active-case counts with rolling-window statistics) improve prediction quality. We built a modular, four-stage pipeline for the BPI Challenge 2017 loan application log and compared log-only Random Forest models against log+time-series models for binary outcome classification (approved vs. not-approved) and remaining-time regression. Time-series features added no meaningful improvement for classification (F1-weighted +0.1%). For regression, time-series features degraded performance (RMSE +1.45). We analyse why time-series information fails to improve either task and discuss implications for feature engineering in predictive process monitoring. The pipeline is open-source and fully reproducible.

Keywords: Predictive Process Monitoring · Process Mining · Time-Series Features · Random Forest · Model Evaluation · Explainability

1 Introduction

Predictive process monitoring (PPM) predicts future states of running process instances from event log data [?]. Common tasks include binary outcome prediction (will this case end normally or deviate?) and remaining-time regression (how many hours until completion?). Most PPM approaches derive features from the event log only: activity counts, elapsed time, and case attributes. These features capture what happened in a case but not the concurrent workload on the process [?]. The BPI Challenge 2017 dataset [?] records a loan application process at a Dutch financial institution, with validation, offer, and approval stages. Resource contention during peak periods can affect both outcomes and processing times, yet log-based features do not capture this workload variation over time.

This project answers: *do aligned time-series workload features improve predictive performance for outcome and remaining-time tasks compared to event-log features alone?* We built a modular pipeline that extracts log-based fea-

tures, computes time-series features, trains Random Forest models, and evaluates them against baselines. The contributions are: an open-source, reproducible PPM pipeline with time-series support, an empirical comparison of log-only vs. log+time-series models on both tasks, and an analysis of when time-series features help and when they do not.

The paper is organised as follows. ?? describes the dataset and preprocessing. ?? covers feature engineering. ?? specifies the models and evaluation protocol. ?? presents results. ?? describes testing evidence. ?? discusses findings and limitations. ?? concludes.

2 Data and Preprocessing

2.1 Event Log: BPI Challenge 2017

The BPI Challenge 2017 event log [?] records a loan application process at a Dutch financial institution. Cases describe the lifecycle of credit applications, including application processing, offer creation, document validation, and final outcomes. Events belong to three categories: application state changes, offer state changes, and workflow activities performed by employees. Each case ultimately ends in either approval (**A_Pending**) or a non-approved outcome (**A_Cancelled** or **A_Denied**). [?]

The dataset is widely used in predictive process monitoring because it contains rich control-flow, temporal, and financial information, as well as substantial variation in case duration and outcome. ?? summarises the dataset.

Table 1: BPI Challenge 2017 dataset statistics.

Statistic	Value
Cases	31,509
Events	1,202,267
Activities	26
Start date	2016-01-01
End date	2017-02-01
Mean case length (events)	38.16
Median case length (events)	35

2.2 Preprocessing Pipeline

Raw XES event logs are first ingested, cleaned, and sorted in ascending order by timestamp. A sliding-window approach is then applied to generate training instances. For each case, a window of length k at time step t consists of the most recent k events ending at position t , where k is constrained by a minimum and maximum window length. Each resulting window is associated with the corresponding case label. Table ?? summarises the final dataset sizes for each

prediction task, together with the applied filtering steps and sliding-window configuration.

The preprocessing pipeline differs between the regression and classification tasks. For the regression task, no additional filtering is applied. Sliding windows are generated with a minimum length of 3 and a maximum length of 100 events per case, resulting in truncated event histories that capture recent temporal context.

For the classification task, additional constraints are introduced to ensure label consistency and prevent information leakage. First, cases that do not contain any of the defined outcome events are removed (98 out of 31,509 cases). Second, windows that include an outcome event are excluded, as their inclusion would leak information about the final case outcome into the input representation. The same minimum and maximum window lengths (3 to 100 events) are used as in the regression setup.

Table 2: Preprocessing summary per task.

Component	Value
Initial dataset	31,509 cases / 1,202,267 events
Window type	Sliding history window (end at t)
Window length	min = 3, max = 100
Regression filtering	None (all cases retained)
Classification filtering	No-outcome cases removed (98 excluded), outcome windows removed
Regression final size	31,509 cases
Classification final size	31,411 cases

2.3 Temporal Train-Test Split

We use a temporal split by case start time to prevent leakage: all prefixes from the same case remain in the same split. Cases starting before the 0.8 quantile go to training. Later cases go to testing. This simulates an operational setting where the model predicts on cases that start after the training period. ?? shows the resulting split statistics.

Table 3: Temporal split statistics.

	Train	Test
Cases	22,859	6,283
Prefixes	763,417	210,096
Split date	2016-01-01	2016-10-18

3 Feature Engineering

3.1 Log-Based Features (Baseline)

Log-based features are derived purely from the event log. ?? lists the feature groups. Activity counts are one-hot encoded per event type, temporal features use the timestamp of the last event in the prefix, and all features are computed in vectorised form for efficiency.

Table 4: Log-based feature groups.

Group	Features	Count
Activity counts	Count per event type per prefix	26
Temporal	Elapsed time, time since last event	2
Financial	Monthly cost, number of terms	2
Waiting states	Time in waiting activities	4
Total		34

3.2 Time-Series Features (Enhanced)

Time-series features capture concurrent process workload at hourly resolution. For every hourly base signal we compute three features: the raw value, an 8-hour trailing mean (`rolling_mean_8h`), and a trend (`trend_8h` = current value minus value 8 hours earlier). ?? lists the time-series feature classes.

Table 5: Time-series feature classes.

Class	Base signal	Base cols	Total
ActiveCaseCount	Cases active at prefix timestamp	1	3
FinancialVolume	Hourly sum/mean of withdrawal, offer, cost	6	18
DecisionRate	Ratio of accepted decisions per hour	1	3
Total		8	24

3.3 Feature Extraction Pipeline

Features are extracted via a modular pipeline. Each feature set implements the `PrefixFeature` protocol (`name()`, `fit()`, `__call__()`, `feature_names`). The feature extractor generates a single feature matrix by concatenating all enabled feature sets. Time-series features are precomputed once during `fit()` and aligned to each prefix at extraction time via vectorised timestamp lookup.

4 Model Specification and Evaluation Protocol

4.1 Prediction Tasks

The project addresses two tasks: binary outcome classification (predict whether a case ends as approved or not-approved) and remaining-time regression (predict hours until case completion). Evaluation metrics are described in ??.

4.2 Models and Hyperparameter Optimisation

Four model families were compared for each task (??). The best model differs by task: HistGradientBoosting (HGB) achieved the highest F1-weighted for classification (0.6601), while Random Forest (RF) achieved the lowest RMSE for regression (228.97). Both are tree-based ensemble methods that handle non-linear relationships and do not require feature scaling. All classification models use `class_weight="balanced"` to address class imbalance.

Table 6: Models and hyperparameter search spaces.

Model	Task	Key hyperparameters
Random Forest	Both	<code>n_estimators</code> : 100–300 <code>max_depth</code> : 5–20 <code>min_samples_split</code> : 2–20
Logistic Regression	Classification	<code>C</code> : 0.01–100
Ridge Regression	Regression	<code>alpha</code> : 0.01–100
HistGradientBoosting	Both	<code>learning_rate</code> : 0.01–0.3 <code>max_iter</code> : 100–300

Hyperparameters are optimised via `RandomizedSearchCV` with 5-fold cross-validation using a sampled subset of training prefixes (`search_sample_size=50 000`) for faster iteration. ?? lists the search configuration.

Table 7: Hyperparameter optimisation configuration.

Parameter	Value
Search method	<code>RandomizedSearchCV</code>
CV folds	3 (5 on dev)
Search iterations	10 (20 on dev)
Search sample size	50 000 prefixes
Scoring	F1-weighted (classification) / RMSE (regression)

4.3 Evaluation Protocol and Comparison Design

We select three metrics for classification. **F1-weighted** balances precision and recall while accounting for class support, making it robust to the 67:33 class imbalance. It is the primary ranking metric used by the pipeline. **ROC AUC** provides a threshold-independent measure of discrimination and is the most commonly reported metric in PPM literature. **MCC** (Matthews Correlation Coefficient) aggregates all four confusion matrix entries into a single score (range -1 to $+1$, where 0 = random) and acts as a balanced tiebreaker when F1 and ROC AUC disagree.

For regression we select **RMSE** in hours (same unit as the target, penalises large errors, primary ranking metric) and R^2 (proportion of variance explained, normalised against the mean baseline).

Every model configuration is trained twice: once with log-only features and once with log-based and time-series features combined. Both configurations share the same train/test split, hyperparameter search space, and evaluation pipeline. The comparison tool (`evaluate_feature_impact.py`) loads both evaluation reports and produces paired metric tables and faceted plots.

5 Results

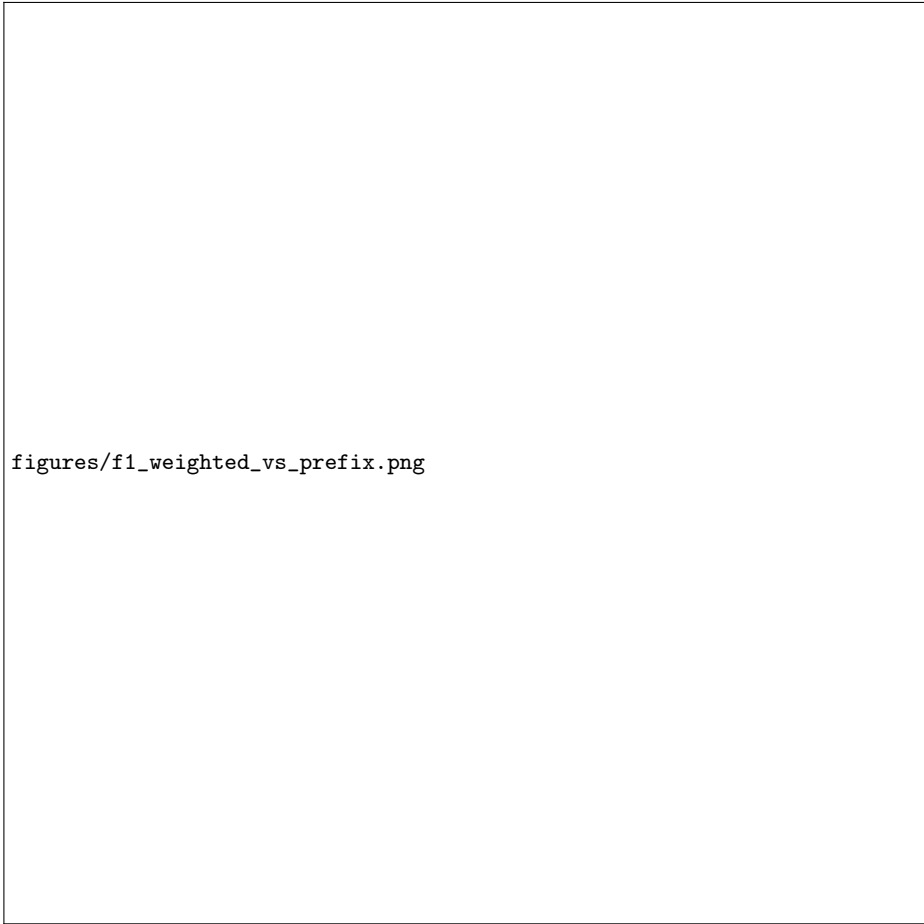
5.1 Outcome Prediction

HistGradientBoosting (HGB) outperforms all alternatives for outcome classification, achieving the highest scores across all three primary metrics (??). HGB leads Random Forest by 2.1% in F1-weighted (0.6601 vs. 0.6465), by 0.006 in ROC AUC, and by 0.019 in MCC.

Table 8: Classification model comparison (log+TS, 58 features, full data).

Model	F1-weighted ROC AUC MCC		
HistGradientBoosting	0.6601	0.6580	0.2316
Random Forest	0.6465	0.6522	0.2128
Logistic Regression	0.6297	0.6270	0.1827
Dummy (most frequent)	0.5401	0.5000	0.0000

?? shows F1-weighted across prefix lengths. Performance improves consistently with more events: HGB rises from 0.5051 at prefix 3 to 0.7731 at prefix 34. The improvement rate diminishes after the first 10–15 events, but longer prefixes continue to add signal.



figures/f1_weighted_vs_prefix.png

Fig. 1: F1-weighted by prefix length (classification).

Time-Series Impact Adding time-series features to HGB changes nothing. With 34 log-only features, HGB scores 0.6584 F1-weighted and 0.6597 ROC AUC. With 58 log+TS features, it scores 0.6594 and 0.6557 (??). The differences (+0.001 in F1-weighted and -0.004 in ROC AUC) are negligible.

Table 9: Classification TS impact (HGB, full data).

Configuration	Features	F1-weighted	ROC AUC
HGB log-only (34)	34	0.6584	0.6597
HGB log+TS (58)	58	0.6594	0.6557

Feature Importance ?? lists the top 10 features for HGB log-only. A single feature dominates: `count_W_Validate application` (importance 0.0636) is three times the second-ranked feature and accounts for the majority of predictive signal.

Table 10: Top 10 features by permutation importance (HGB log-only).

Feature	Importance
count_W_Validate application	0.0636
count_W_Handle leads	0.0209
monthly_cost	0.0123
num_terms	0.0097
elapsed_time_hours	0.0074
count_A_Validating	0.0061
count_O_Returned	0.0060
count_W_Call after offers	0.0057
count_W_Call incomplete files	0.0043
time_since_last_event_hours	0.0023

?? visualises the importance distribution: activity-count features occupy 6 of the top 10 positions, while financial and temporal features appear at middle ranks with substantially lower importance and waiting-state features do not appear in the top 10 at all.

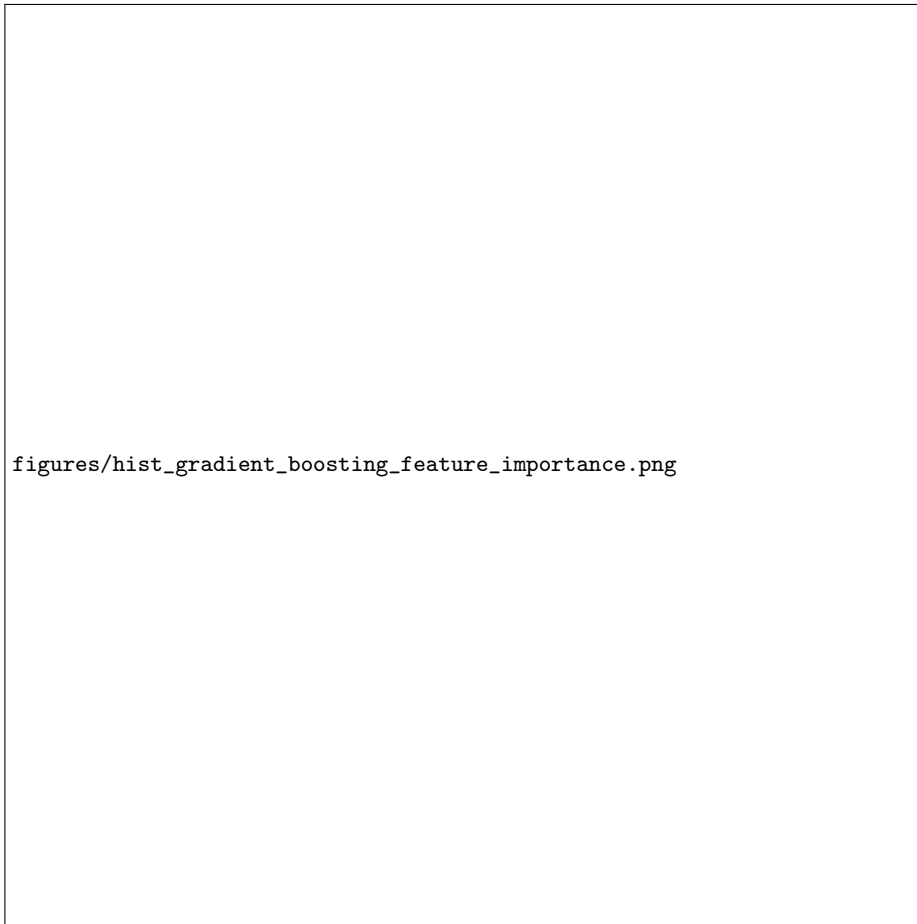


Fig. 2: Permutation feature importance (HGB log-only).

?? shows the SHAP summary dot plot for the same model, revealing the direction of each feature's effect. High occurrences of **count_W_Validate application** (the dominant predictor) push predictions toward the approved class. Conversely, higher **elapsed_time_hours** and **time_since_last_event_hours** push toward not-approved, suggesting that longer-running cases are more likely to end negatively.

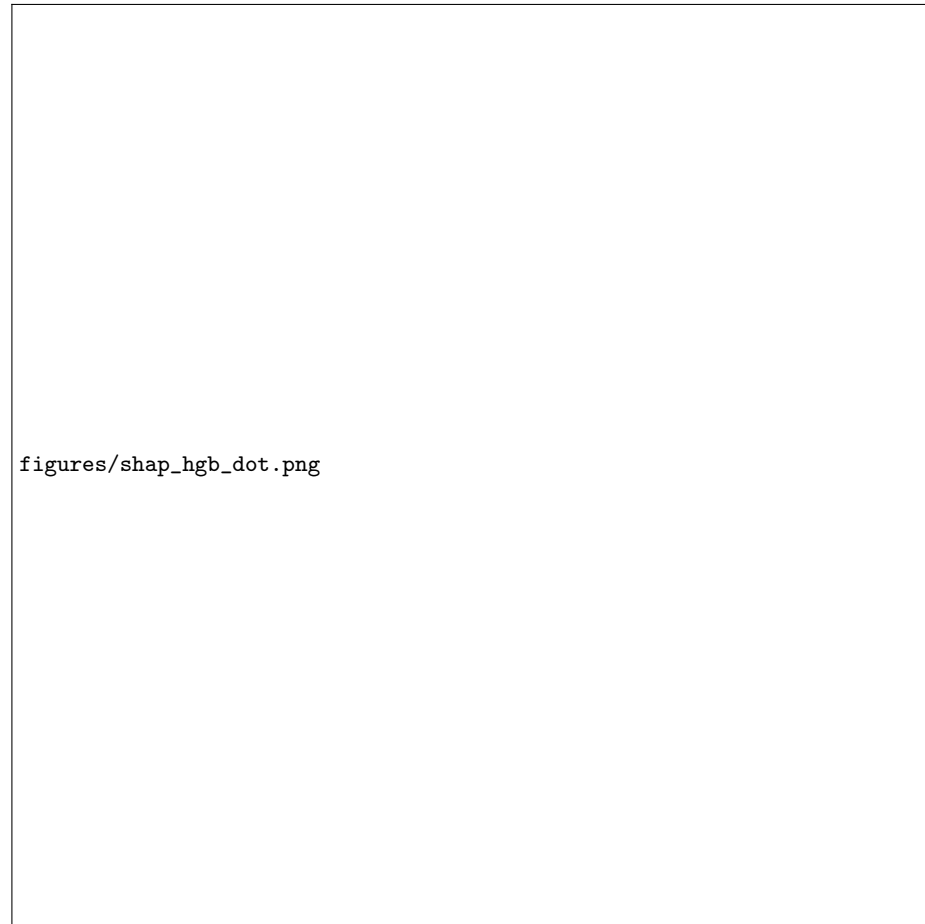


Fig. 3: SHAP summary dot plot (HGB classification). Each dot represents a single prediction. Colour indicates feature value (red = high, blue = low). Features are ordered by global importance.

5.2 Remaining-Time Prediction

Random Forest (RF) achieves the lowest RMSE for remaining-time regression, marginally ahead of HGB and well below the dummy baseline (??). The R^2 of 0.382 indicates that RF explains 38% of the variance in remaining time.

Table 11: Regression model comparison (log+TS, max_length=100, full data).

Model	RMSE (hours)	R^2
Random Forest	230.28	0.382
HistGradientBoosting	231.15	—
Ridge	237.33	—
Dummy (mean)	295.97	0.000

?? shows RMSE across prefix lengths. RMSE improves from 284.83 at prefix 3 to 233.14 at prefix 100, with most of the gain achieved by prefix 20. Beyond that, returns diminish but do not fully plateau.

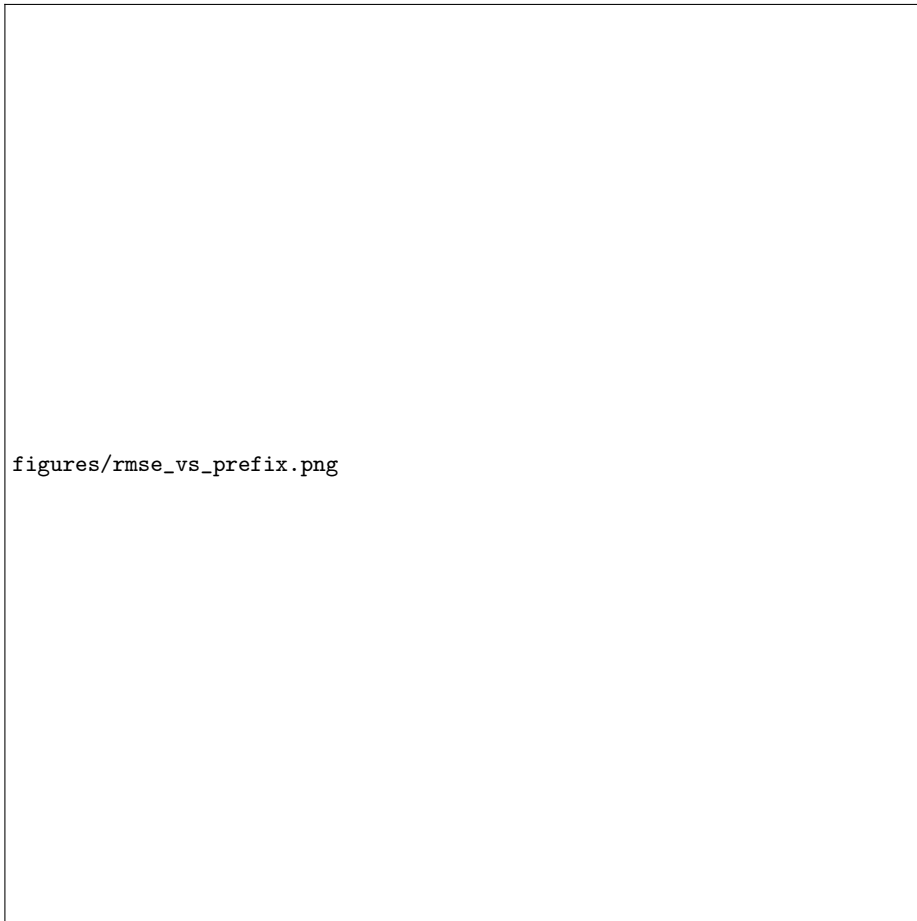


Fig. 4: RMSE by prefix length (RF log-only regression).

Time-Series Impact Time-series features degrade regression performance. The log-only RF (228.97 RMSE) beats every log+TS variant by 1.31–1.45 RMSE (??), a 0.6% penalty. The degradation is consistent across all prefix lengths.

Table 12: Regression TS impact (RF, max_length=100, full data).

Configuration	Features	RMSE (hours)
RF log-only (34)	34	228.97
RF log+TS (58)	58	230.42
RF log+TS (compare)	58	230.28

Full-Length vs. Maximum-Length Prefixes Capped (max_length=100) and unbounded (full trace) prefix configurations produce nearly identical results: 230.28 vs. 230.20 RMSE. A difference of 0.04% indicates that longer histories do not help.

Feature Importance ?? lists the top 10 features for RF log-only. count_W_Validate application dominates here as well (0.0618), with count_W_Call after offers (0.0323) and count_A_Cancelled (0.0299) distantly behind, with activity counts filling 7 of 10 positions and temporal features carrying moderate importance.

Table 13: Top 10 features by permutation importance (RF log-only).

Feature	Importance
count_W_Validate application	0.0618
count_W_Call after offers	0.0323
count_A_Cancelled	0.0299
count_A_Validating	0.0132
elapsed_time_hours	0.0131
count_O_Returned	0.0127
count_A_Complete	0.0111
time_since_last_event_hours	0.0096
count_O_Accepted	0.0068
num_terms	0.0048

5.3 Overfitting Analysis

?? reports train-test gaps for the best models. ?? visualises the comparison.

Table 14: Train-test performance gap.

Task	Model	Train	Test (gap)
Classification	HGB (f1-weighted)	0.6971	0.6609 (−0.0362)
Classification	RF (f1-weighted)	0.6810	0.6510 (−0.0300)
Regression	RF log-only (RMSE)	248.08	233.14 (−14.94)

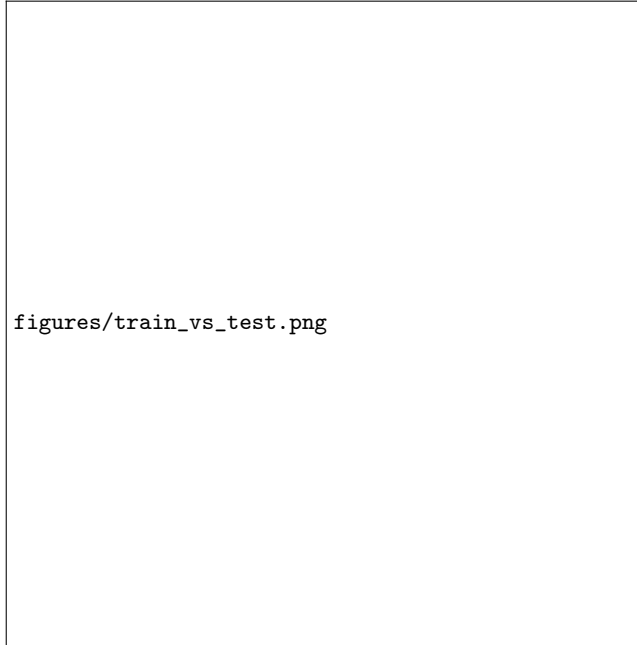


Fig. 5: Train vs. test performance (all models).

Overfitting is moderate: the train-test gap for HGB is -0.036 F1-weighted, while for RF regression the gap is -14.94 RMSE (6% of the test RMSE), both within expected bounds for tree-based ensemble models without explicit regularisation.

5.4 Summary of Findings

- **HGB is best for classification** (0.6601 F1-weighted), outperforming RF by 2.1%.
- **RF is best for regression** (228.97 RMSE log-only), marginally ahead of HGB.

- **Time-series features add no value for classification** (+0.1% F1-weighted) and **harm regression** (+1.45 RMSE).
- **Unbounded prefixes yield no improvement** over `max_length=100` (−0.04% RMSE).
- `count_W_Validate application` is the single most predictive feature across both tasks (importance 0.0636 classification, 0.0618 regression).
- Performance improves with longer prefixes, but the improvement rate diminishes after 10–15 events.
- Overfitting gaps are moderate (HGB −0.036, RF −14.94 RMSE).

6 Testing Evidence

The project includes 139 unit and integration tests covering:

- Label construction on synthetic event logs.
- Prefix generation: number of prefixes, ordering, case identity.
- Temporal split: no case overlap between train and test.
- Feature matrix shape, column names, and zero-variance detection.
- Dynamic prefix-length column handling in evaluation.
- Model training smoke tests on synthetic data.
- Metric computation correctness (MAE, RMSE, F1, AUC-ROC).

Integration tests mock the dataset download to avoid external dependencies. All tests pass with a single command:

```
poetry run pytest -m "not integration"
```

7 Critical Evaluation

7.1 Do Time-Series Features Improve Predictive Performance?

For classification, time-series features added no meaningful improvement (F1-weighted +0.1%). For regression, time-series features consistently degraded performance (RMSE +1.45), with the log-only model outperforming all log+TS variants across all metric dimensions.

Three factors explain the limited impact. First, the **proxy effect**: log-based features (activity counts, elapsed time) already contain indirect workload information: a high count of `W_Validate application` events correlates with periods of high load. Second, **coarse resampling**: the hourly active-case count loses sub-hour workload variation, and finer-grained features such as per-resource load might capture more signal. Third, **noise for regression**: the remaining-time target is inherently noisy, and adding time-series features introduces variance that harms the already challenging regression task.

7.2 Generalisability, Limitations, and Threats to Validity

Several limitations affect the interpretation of these results. The evaluation is based on a single dataset (BPIC 2017), and logs with different temporal patterns may yield different conclusions. Only one time-series feature family (active-case count with rolling statistics) was evaluated. Other features such as activity queue length, resource utilisation, and event throughput remain unexplored. The pipeline evaluates Random Forest and HistGradientBoosting as tree-based baselines, and deep learning models (LSTMs, transformers) may integrate time-series information differently. The temporal split is deterministic, and time-series cross-validation could provide a more rigorous evaluation.

Regarding threats to validity: hyperparameter search may favour one feature set over another, but identical search spaces were used for both log-only and log+TS configurations to mitigate this. The single dataset limits generalisability (external validity). Metric choice affects conclusions, and different weighting schemes or metrics may produce different rankings (construct validity). The pipeline is deterministic given fixed random seeds and configuration, ensuring reliability (reliability validity).

8 Conclusion

8.1 Summary

This project investigated whether aligned time-series workload features improve predictive process monitoring. On the BPI Challenge 2017 dataset, time-series features added no meaningful improvement for classification (F1-weighted +0.1%) and degraded remaining-time regression (RMSE +1.45). These results suggest that log-based features may capture workload variation indirectly: the dominance of `count_W_Validate application` across both tasks is consistent with a proxy effect, where activity counts of gate events correlate with concurrent process load. In this setting, time-series features added negligible signal beyond what the log already provides.

Data and Code Availability

The source code is available at <https://github.com/durki007/spi-time-series>. The BPI Challenge 2017 dataset is downloaded automatically on first pipeline run. To reproduce the experiments:

```
pip install -r requirements.txt
python -m spi_time_series.main experiments/classification/classification_compare.yaml --output-dir results
python -m spi_time_series.main experiments/regression/regression_rf_log.yaml --output-dir results
```

A Appendix: Additional Results

A.1 Classification: Full Model Comparison

Table 15: Classification: full metrics for all models (log+TS, 58 features).

Model	F1-weighted ROC	AUC	MCC
HGB	0.6601	0.6580	0.2316
RF	0.6465	0.6522	0.2128
LR	0.6297	0.6270	0.1827
Dummy	0.5401	0.5000	0.0000

A.2 Regression: Full Model Comparison

Table 16: Regression: full metrics for all models (log+TS, max_length=100).

Model	RMSE	R^2
RF	230.28	0.382
HGB	231.15	—
Ridge	237.33	—
Dummy	295.97	0.000

A.3 TS Impact Summary

Table 17: TS impact summary across all configurations.

Task	Model	Features	F1 / RMSE	ROC AUC / R^2
Classification	HGB log-only	34	0.6584	0.6597
Classification	HGB log+TS	58	0.6594	0.6557
Regression	RF log-only	34	228.97	0.388
Regression	RF log+TS	58	230.42	0.381
Regression	RF log+TS (full)	58	230.20	0.382

A.4 Overfitting Details

Table 18: Train-test comparison for all models.

Task	Model	Metric	Train	Test
Classification	HGB	F1-weighted	0.6971	0.6609
Classification	RF	F1-weighted	0.6810	0.6510
Classification	LR	F1-weighted	0.6563	0.6388
Classification	Dummy	F1-weighted	0.5454	0.5401
Regression	RF	RMSE	248.08	233.14
Regression	HGB	RMSE	238.74	234.88
Regression	Ridge	RMSE	256.93	240.87
Regression	Dummy	RMSE	295.97	295.97

B Appendix: Configuration Files

Experiments are defined in YAML configuration files under `experiments/{task}/`. Key parameters: `data.split_quantile=0.8`, `prefix.min_length=3`, `search.cv_folds=3`, `features.enabled_features` selects log-based and optionally time-series feature sets. See the `experiments/` directory for complete configurations for each experiment.

C Appendix: Feature Descriptions

C.1 Log-Based Features (34 features)

- **ActivityCountFeatures (26)**: One count per activity label (e.g., `count_W_Validate` application, `count_A_Accepted`). Each feature records how many times the event type occurred in the current prefix.
- **TemporalFeatures (2)**: `elapsed_time_hours` (hours since case start) and `time_since_last_event_hours` (hours since the most recent event).
- **FinancialFeatures (2)**: `monthly_cost` and `num_terms` from the loan application’s offer.
- **WaitingStateFeatures (4)**: `hours_since_offer_sent`, `hours_since_incomplete`, `is_waiting_for_customer`, `is_waiting_for_bank`.

C.2 Time-Series Features (24 features)

Each base signal produces three columns: raw value, 8-hour trailing mean (`window_mean_8h`), and 8-hour trend (`trend_8h` = current value minus value 8 hours earlier).

- **ActiveCaseCountFeature (1 base, 3 total)**: Number of cases active at the prefix’s timestamp in the hourly active-case time series.

- **FinancialVolumeFeature (6 base, 18 total)**: Hourly aggregates: total and mean of withdrawal amount, offer amount, and monthly cost.
- **DecisionRateFeature (1 base, 3 total)**: Ratio of accepted decisions (events 0_Accepted) per hour.